
Titulo del TFG
Title in english



Trabajo de Fin de Grado
Curso 2025–2026

Autor
Eva Paula Mik

Director
Ismael Sagredo Olivenza

Colaborador
Colaborador no definido. Usa \colaboradorPortada

Grado en Desarrollo de Videojuegos
Facultad de Informática
Universidad Complutense de Madrid

Titulo del TFG
Title in english

Trabajo de Fin de Grado en Desarrollo de Videojuegos

Autor
Eva Paula Mik

Director
Ismael Sagredo Olivenza

Colaborador
Colaborador no definido. Usa \colaboradorPortada

Convocatoria: *Junio 2026*

Grado en Desarrollo de Videojuegos
Facultad de Informática
Universidad Complutense de Madrid

17 de abril de 2026

Dedicatoria

*A toda la gente que nos ha hecho llegar hasta
aquí,
un besazo.
A todos los que han estado a nuestro lado.*

Agradecimientos

Os amo!

Mik

Os amo!

Pau

Os amo!

Eva

Resumen

Titulo del TFG

Nuestro estudio se basa en la comparativa de distintos modelos de IA para la frente a la simulación de físicas costosas en proyectos 3D en motores de videojuegos. Para ello creamos una simulación en Unity con un objeto con componente Cloth y distintos colliders que interactuasen con ella. Tras generar los datasets, probamos distintos modelos de IA e integramos los datos. Finalmente exportamos a unity con Sentis y realizamos comparativas teniendo en cuenta GPU, CPU. Observamos que el modelo que mejores resultados obtuvo en cuanto a tal métrica fue X.

Palabras clave

Inteligencia Artificial, Simulación física, Unity, Videojuegos, Optimización, PyTorch, ML

Abstract

Title in english

We need to do this.

Keywords

We need to do this.

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Plan de trabajo	2
1.4. Metodología	4
1.5. Estructura de la memoria (propuesta)	4
2. Estado de la Cuestión	5
2.1. Simulación física en videojuegos	5
2.1.1. Motores de física	6
2.1.2. Motores de videojuegos	6
2.2. Simulación de telas	7
2.2.1. ¿Qué es Cloth?	7
2.2.2. El problema de optimización	7
2.2.3. Soluciones	8
2.3. Inteligencia Artificial en videojuegos	9
2.3.1. Tecnologías existentes	9
2.3.2. Librerías	9
2.3.2.1. Pytorch	9
2.3.2.2. Tensorflow	9
2.3.2.3. Sentis	9
2.3.3. Modelos de optimización de telas	9
2.3.3.1. Subspace Neural Physics: Fast Data-Driven Interac- tive Simulation	9
2.3.3.2. Neural Cloth Simulation	9
2.3.3.3. PBNS: Physically Based Neural Simulation for Un- supervised Garment Pose Space Deformation	9
2.3.3.4. HOOD: Hierarchical Graphs for Generalized Mode- lling of Clothing Dynamics	9
2.3.3.5. Cloth and Skin Deformation with a Triangle Mesh Based Convolutional Neural Network	10
2.3.4. DeepSD: Real-time Cloth Simulation with Deep Learning	10

3. Descripción del Trabajo	11
3.1. Búsqueda de un dataset	11
3.1.1. Dataset de la Universidad Carnegie Mellon	11
3.1.2. Kaggle	12
3.2. Aplicación final	13
3.2.1. Funcionamiento de la aplicación	14
4. Conclusiones y Trabajo Futuro	17
4.1. Conclusiones	17
4.1.1. Conclusiones de la búsqueda de datasets	17
4.1.2. Conclusiones de la blablabla	17
4.1.3. Limitaciones	17
4.2. Trabajo Futuro	17
Contribuciones Personales	19
Bibliografía	21
A. Carteles para la generación del dataset	23
B. Formulario de recogida de citas	27
C. Resultados de los distintos modelos CNN	29
C.1. Intervalo 0.2s	29
C.2. Intervalo 0.4s	31
C.3. Intervalo 0.6s	32
C.4. Intervalo 0.8s	34
C.5. Intervalo 1.0s	35
Glosario	37
Licencia de uso del documento	39
Licencia de uso del código fuente	41

Índice de figuras

1.1. Diagrama de Gantt del proyecto	4
2.1. Imágen del traje de captura de movimiento XSens	9
3.1. Imágen del traje de captura de movimiento utilizado para la base de datos de la Universidad Carnegie Mellon (izquierda) y el esqueleto resultante (derecha). Fuente: https://mocap.cs.cmu.edu/info.php	12
3.2. Captura de la aplicación final desde el editor de Unity	14
3.3. Diagrama de flujo de la aplicación de demostración	16
A.1. Cartel colgado en la facultad de informática	23
A.2. Cartel colgado en redes sociales	24
A.3. Cartel colgado en pantallas de la facultad	25
B.1. Formulario de recogida de citas (primera parte)	27
B.2. Formulario de recogida de citas (segunda parte)	28
C.1. Esquema del modelo CNN con 0.2s de intervalo	29
C.2. Matriz de confusión del modelo CNN con 0.2s de intervalo	30
C.3. Gráfico de entrenamiento del modelo CNN con 0.2s de intervalo (me- jor val_accuracy = 0.28777)	30
C.4. Esquema del modelo CNN con 0.4s de intervalo	31
C.5. Matriz de confusión del modelo CNN con 0.4s de intervalo	31
C.6. Gráfico de entrenamiento del modelo CNN con 0.4s de intervalo (me- jor val_accuracy = 0.49473)	32
C.7. Esquema del modelo CNN con 0.6s de intervalo	32
C.8. Matriz de confusión del modelo CNN con 0.6s de intervalo	33
C.9. Gráfico de entrenamiento del modelo CNN con 0.6s de intervalo (me- jor val_accuracy = 0.44952)	33
C.10. Esquema del modelo CNN con 0.8s de intervalo	34
C.11. Matriz de confusión del modelo CNN con 0.8s de intervalo	34
C.12. Gráfico de entrenamiento del modelo CNN con 0.8s de intervalo (me- jor val_accuracy = 0.40583)	35
C.13. Esquema del modelo CNN con 1.0s de intervalo	35

C.14. Matriz de confusión del modelo CNN con 1.0s de intervalo	36
C.15. Gráfico de entrenamiento del modelo CNN con 1.0s de intervalo (mejor val_accuracy = 0.56002)	36

Índice de tablas

3.1. Reacción del npc al gesto predicho del jugador	15
---	----

Introducción

En los últimos años el aumento de tamaño y complejidad en los juegos ha supuesto la necesaria optimización y automatización (pruebas de software con GameDriver, niveles procedurales) de sus partes, tanto en render como en lógica. Particularmente, en el aspecto de render, cada vez se tiende a gráficos más detallados y simulaciones más complejas en entornos 3D. Cosas como el pelo, fluidos o ropa suponen una carga de cálculos para la ejecución de los juegos. Cada vez más estas partes se están calculando mediante ML y DL para acelerar el proceso y evitar la caída de frames. Empresas pioneras en hardware y software han desarrollado tecnologías para el aumento de rendimiento de juegos incorporando algoritmos de ML y DL, como por ejemplo: NVIDIA con el DLSS (Deep Learning Super Sampling) y AMD con FSR (FidelityFX Super Resolution). Juegos AAA como el Resident Evil Requiem usa DLSS 4. Otras empresas como EA (FIFA) o Embark Studios (Arc Raiders), lo incorporan en el gameplay para simular comportamiento. Ubisoft tiene múltiples líneas de investigación con IA, en las que destacaremos las de renderizado y simulación física. Insértense ejemplo de empresa que ha aplicado en sus juegos ML

1.1. Motivación

Estas simulaciones no sólo se dan en consolas con gráficas dedicadas, sino que también se llevan a cabo en máquinas de menor potencia, y cada vez más en procesadores modernos que ofrecen ventajas para los procesadores neuronales (What does this mean?). Cada juego, búsqueda o motor posee sus técnicas para aprovechar recursos y no ralentizar los frames)? Podemos observar que empresas como Ubisoft (citar) están investigando la optimización simulación de telas con ML (neuronas simples). Observamos que por ejemplo, motores de videojuegos como Unity, utilizado por más desarrolladores indie, no utiliza la GPU para calcular físicas en estos casos, lo cual supone que animaciones y físicas más complejas sean incompatibles para ejecutar (forzando a los desarrolladores, por ejemplo, a simplificar sus modelos y animaciones). Creemos que incorporar la IA puede ser clave para este reto técnico, y permitirá agilizar el proceso en máquinas de menor capacidad. (Este gasto de recursos y memoria (entrenamiento) inicial conlleva una bajada de frames en ejecución). Si bn utilizar la IA es sacrificar q las físicas sean super precisas, creemos q esto es caer

en la falacia d la inmersión Hacer más accesible las físicas complejas/animaciones en máquinas poco potentes como concepto? Justificamos el haber elegido cloth como experimento de diseño, para trabajar cn la percepción. ante el intento de la industria de conseguir físicas cada vez más realistas en un intento de hacer q la ux se sienta más real, queremos conseguir mejores simulaciones cn menos carga computacional pero sin caer en la falacia de la inmersión referencia d videojuegos al mítico libro d diseño d Rules of Play, y q nuestro objetivo es crear simulaciones físicas realistas sin fijarnos tanto en q sean absolutamente precisas. Creo q puede ser interesante para darle una vuelta d game developers y tmbn si a nivel ia la física no da para mucho le damos un aproach más d diseño de videojuegos / experimento d percepción ..

1.2. Objetivos

El objetivo general de este proyecto es la implementación de un modelo de IA que, con baja latencia, permita replicar el movimiento en tiempo real de una tela (física compleja) en un motor de videojuegos. Para cumplir el objetivo, se han propuesto las siguientes metas durante el desarrollo del trabajo:

1. Generación de una animación con una tela y un colisionador que varíe en cada ejecución.
2. Generación, extracción del motor y procesamiento de datasets con la información relevante de la tela respecto al entorno y colisionador.
3. Implementación, entrenamiento y comparación de distintos modelos de IA, tanto redes neuronales (LSTM, CNN y RNN, GNN?) como otros modelos (no creo) y su integración en el motor de videojuegos.
4. Pruebas visuales de los distintos modelos, análisis?

1.3. Plan de trabajo

El desarrollo de este proyecto se llevará a cabo en el periodo lectivo del curso 2025-2026, y dados los objetivos del apartado anterior el equipo plantea dividir el trabajo en los siguientes en 5 apartados:

- **Iteración 0:** (..oct) Antes de empezar, la propuesta, que en un principio se plantea como optimizar simulaciones físicas con inteligencia artificial, ha de ser más precisa. Es en esta etapa donde se concretan los objetivos reales de la investigación y se planea el proyecto.
- **Iteración 1:** Ya con una idea y unos objetivos concretos, empezará la etapa de preparación. En esta etapa se cumplirán los dos primeros subobjetivos o metas, consiguiendo así la implementación de un sistema de simulación listo para ser posteriormente procesado por los modelos.

- **Iteración 1.1:** (octfeb) La investigación de trabajos relacionados con el nuestro será indispensable para dirigir el desarrollo de nuestros experimentos. Se analizarán distintas alternativas para decidir cuales nos resultará más interesante replicar, se definirá que tipo de datos debere-mos recopilar para el entrenamiento y que tipo de redes podrán llevarlo a cabo.
 - **Iteración 1.2:** (enemar) Se creará una simulación básica en Unity y un sistema de registro de los datos que nos interesen de la tela.
-
- **Iteración 2:** (mar) Se establecerá un entorno de trabajo para el entrenamiento de los datos recopilados. Se generará un pipeline claro en el que se integre la simulación de recogida de datos, su limpieza y posterior uso para entrenar el modelo, y finalmente la incorporación de este al entorno de simulación. Para poder asegurarnos del correcto funcionamiento de este sistema se generará un modelo simple y se comprobará que el pipeline es correcto. En esta fase, se comenzarán a redactar los primeros capítulos de la memoria.
 - **Iteración 3:** (abril) Con una base sólida ya montada, se aumentará la complejidad y por consiguiente el coste de la simulación. Se implementarán distintos modelos y se llevarán a cabo pruebas con todos ellos para asegurarse de su fiabilidad. Así mismo se seguirá adelante reflejando todos estos avances en el contenido de la memoria, y cerrando los primeros capítulos de la misma.
 - **Iteración 4:** (abril-mayo) Si las anteriores iteraciones funcionan de manera correcta esta última no tendrá ninguna carga de implementación. Se alcanzará la última meta definida; que consistirá en hacer distintas pruebas entrenando los modelos ya desarrollados, sacar métricas de todos ellos para poder hacer comparaciones y hacer un análisis exhaustivo de estos resultados, sacando así las conclusiones de la investigación. Por supuesto se acabará la memoria y se preparará la defensa del trabajo.

El desarrollo de las tareas y su planificación se puede ver en la figura [1.1](#).

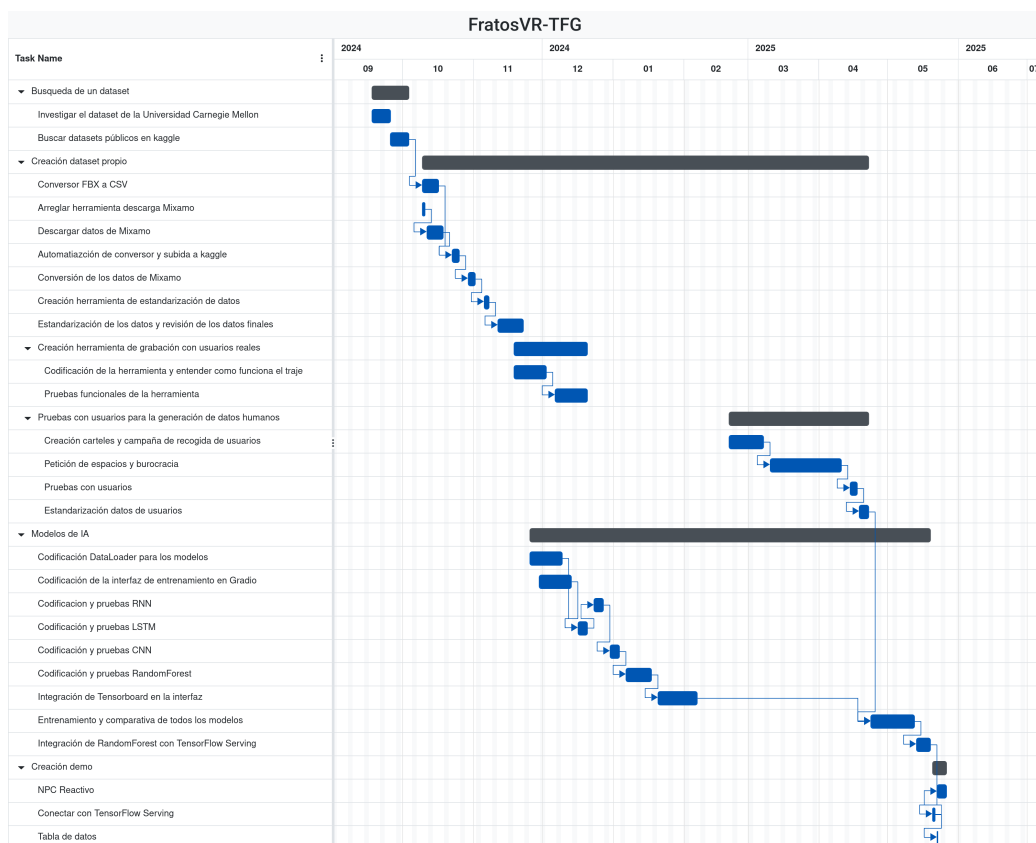


Figura 1.1: Diagrama de Gantt del proyecto

1.4. Metodología

Para el desarrollo de esta investigación, se utilizarán una serie de herramientas de libre acceso para todos los públicos y gratuitas, lo que facilita la reproducibilidad de los resultados obtenidos. Desarrollo en SO Windows 11, lo más común en nuestro equipo y posibles usuarios? Creación de la animación e integración del modelo en Unity versión 6000.3.2f1 (última versión disponible de soporte a largo plazo). Lenguaje de programación: C en unity para la recogida de datos y utilización del modelo, python para el entrenamiento. Sentis como integración del modelo entrenado fuera en code. Github para trabajo en equipo (basado en Git, control de versiones). Github projects para la creación y asignación de tareas en iteraciones. Google Drive como herramienta adicional de almacenaje para apuntes de reuniones, pasos que damos. Discord para llevar a cabo reuniones, a parte de las reuniones presenciales semanales.

1.5. Estructura de la memoria (propuesta)

Capítulo 2

Estado de la Cuestión

En este capítulo se presenta una breve descripción del desarrollo de las simulaciones físicas en videojuegos, así como el hardware y software que ha hecho que esta evolución sea posible, se presenta la simulación de telas en específico, las maneras que existen de implementarla, los problemas de optimización que estas suponen y los avances en modelos de IA que permiten imaginar otras soluciones.

2.1. Simulación física en videojuegos

Basta con remontarse a la creación del Pong en 1972, para darse cuenta de la importancia que las simulaciones físicas han tenido en el ámbito de los videojuegos desde el primer momento. Al fin y al cabo, las físicas permiten crear situaciones del gameplay que no están predefinidas, enriqueciendo así la experiencia del jugador.

En los inicios del desarrollo de videojuegos, las técnicas más rudimentarias para implementar estas físicas incluían:

- **La prerenderización:** Dada la capacidad de computo con la que solían contar las máquinas, era imposible hacer todos los cálculos físicos en tiempo real, por lo que las interacciones físicas podían diseñarse previamente para parecer realistas. [Templet \(2021\)](#)
- **La implementación forzada de cada elemento:** El comportamiento de cada elemento podía ser implementado únicamente para él, en vez de seguir unas normas físicas generales. Por ejemplo, en el caso de un proyectil, podía codificarse calculando su trayectoria y velocidad en pocas líneas de código.

Estos métodos eran claramente limitados en la precisión que ofrecían, y quedaba clara la necesidad de una solución más general y reproducible. El mismo Ian Millington menciona en su libro *Game Physics Engine Development* [Millington \(2007\)](#) el juego *Exile*, creado por Peter Irvin y Jeremy Smith. Si bien este juego fue publicado en 1988 ya contaba con colisiones con intercambio de impulso"(esto no se entiende

igual?), gravedad, fuerzas como por ejemplo el viento, explosiones y etc. Es probablemente el primer juego que cuenta con un motor de físicas completo.

.....incluir página web en la q el propio Peter explica esto y dice q Jeremy se muere? <https://inventivity.co.uk/exileami/about.htm> Dramatismo. Intensidad narrativa a la memoria. Mirar incluir algo más concreto d juegos? Paul et al. (2012)

2.1.1. Motores de fisica

De todas formas, con la introducción del 3D estos métodos carecen de COHERENCIA, la cual es imprescindible para la inmersión del jugador. Hecker (2000) Con la introducción del 3D, subió el nivel. La coherencia necesaria para la inmersión creció exponencialmente, lo que se busca ahora (hiperrealismo tal) y la libertad que se le quiere dar al jugador en la experiencia de juego, por lo que se le ha dado gran importancia a las cosas real-time. Although conventional wisdom in the game industry says the goal of the games is reality itself, especially as infinitely cheap and powerful computers become commonplace, the real goal is consistency. Reality is boring; escaping reality is why people play computer games in the first place. Consistency, on the other hand, is crucial for keeping players immersed in game environments; it's okay to be able to run 100 miles per hour and jump 20 feet as long as the rest of the world reacts appropriately. In effect, each computer game makes a contract with its players when they start playing; any inconsistency violates this contract, reminding the player it's just a game. "Getting stuck in a wall due to bad collision detection is not something a player wants to happen when adventuring in a dungeon. When inconsistencies occur like when a boom microphone floats into a movie frame during an emotional scene the creators of immersive environments have failed.

PHYSIS HAVOC BULLET intereses por trabajar en la CPU vs GPU

2.1.2. Motores de videojuegos

Unity, Unreal Engine and custom game engines of large AAA studios continue to dominate the game engine market. <https://app.sensortower.com/vgi/assets/reports/TheBigGameEng>

Breve definicion d lo q es un motor? Estos, se dividen en componentes y vamos a prestar atención a la evolución del componente encargado de la computación de calculos físicos.

Los dos principales motores para el desarrollo de videojuegos son: hoy en día unreal..... Unreal es un motor de videojuegos creado por Epic Games en 1998 con el lanzamiento de Unreal Engine 1. La versión estable actual es Unreal Engine 5.5, lanzada en noviembre de 2024. El código fuente de este motor está escrito en C++ y es accesible desde Github ¹. Usa PhysX igualmente así que Unreal. Ahora chaos cloth. Comentaremos más adelante.

Hasta esta última versión (5.7) unreal usaba PhysX. Al igual que lo hace unity Built-in 3D physics: Uses an integration of the Nvidia PhysX engine by default.

¹Enlace a la página de Github de Epic: <https://github.com/epicgames>

<https://docs.unity3d.com/Manual/physics-integrations.html> y UNITY (justificamos por qué unity: libre acceso, motor principal usado para desarrollo?) Unity es un motor de videojuegos creado por Unity Technologies en 2005. Su motor físico: PhysX, creado por la empresa Nvidia Corporation.

2.2. Simulación de telas

Entendemos los objetos deformables o soft-bodies como aquellos que no son rígidos. Así como en los objetos rígidos podemos simular su movimiento sabiendo su motion y centro de masa, para los objetos deformables necesitamos tener en cuenta todas las partículas y propiedades físicas que lo componen. Entre estos encontramos elementos como las telas, el agua o el viento [Kenny Erleben \(2005\)](#).

2.2.1. ¿Qué es Cloth?

Cloth es un componente que nos proporciona Unity3D que funciona junto al Skinned Mesh Renderer para simular telas. Está basado en Cloth de PhysX y fue incorporado con el objeto de poder disponer de ropa para personajes. Tiene múltiples propiedades como la resistencia a la flexión, rigidez, damping, uso de gravedad...

Se trata la tela como una malla de partículas conectadas por restricciones. El Skinned Renderer calcula las transformaciones basándose en una malla de baja resolución sobre la que realiza los movimientos en los vértices, que dinámicamente se aplican en la malla real y obtenemos la simulación visual. Un constraint de distancia máxima 0 significa que está fijo en el espacio y que no se calculan sus deformaciones. La cloth cuenta con capacidad de colisionar internamente y con dos tipos de colliders externos: cápsulas o esferas. Cualquier objeto externo con el que se quiera colisionar tendrá que ser un conjunto de estos colliders.

Un solver es un algoritmo computacional que se encarga de predecir el comportamiento del objeto deformable en renderizados gráficos y simulaciones físicas. Encontramos infinitos tipos de solvers. Los primeros solvers se basan en algoritmos que emplean la fuerza bruta: realizan cálculos en masa para sistemas de muelles (MSS). Los solvers más actuales resultan en el PBD proceso iterativo que proyecta las posiciones de las partículas sobre un "espacio de restricciones" [Müller et al. \(2007\)](#).

El solver de PhysX se basa en PBD, es decir, trabaja directamente sobre las posiciones. Encontramos también XPBD [Macklin et al. \(2016\)](#), una versión extendida que puede incorporarse en proyectos.

2.2.2. El problema de optimización

La simulación de cloth en tiempo real es un problema recurrente, que aún continúa buscando la optimización a día de hoy. Debido a la alta carga computacional, los frames decaen inmediatamente en cuanto se complica o aumenta la geometría

de la tela. Esto dificulta enormemente el realismo, y resulta un gran impedimento en entornos dónde se requiere un alto framerate como son los juegos VR frente al motion sickness.

Mientras que el Cloth de PhysX puede delegar la simulación a GPUs compatibles con CUDA, el Cloth integrado por defecto en Unity realiza todo su trabajo en la CPU, lo cual ralentiza mucho la ejecución.

2.2.3. Soluciones

Encontramos varios tipos de soluciones para la optimización de la simulación de telas:

- Optimización por CPU: encontramos la extensión de Obi Cloth [Method \(2019\)](#), que aprovecha el sistema de Jobs de Unity para paralelizar la simulación y acelerar la simulación sin salir de la CPU. Es uno de los paquetes más populares para solucionar este problema.
- Paralelización en GPU: a través de shaders se delega la fuerza de cálculo del skinning en la GPU. Encontramos diferentes estudios que aportan diferentes enfoques para la optimización: Jim Hugunin realiza una investigación para alcanzar altas resoluciones y evitar colisiones en las mallas [Jim Hugunin \(2016\)](#), mientras que otros se centran en la mejora de rendimiento pura para adaptar simulaciones a entornos VR [Va et al. \(2021\)](#).
- Entrenamiento con IA: el enfoque más actual de la resolución de simulaciones complejas como Cloth es la Inteligencia Artificial. Se entrenan modelos de ML y DL con los datos de las mallas animadas o en simulación, con lo que obtenemos más adelante mejor rendimiento en tiempo real. Se sacrifica memoria y tiempo de entrenamiento para obtener mejores y mucho más rápidas simulaciones de tela en escena. Actualmente, nos encontramos en el paso de investigación a implementación en grandes empresas: Unreal ya proporciona una simulación de tela a través de aprendizaje automático [Games \(2025\)](#), en el que destaca su realismo. Aún así, cuenta con limitaciones al basarse en NearestNeighborSearch: el sistema es capaz de predecir detalles como arrugas pero no movimiento dinámico como ondulaciones o drapeados. En la próxima sección detallaremos las numerosas investigaciones recientes que abarcan aspectos desde la reproducción de arrugas, el compromiso entre calidad, coste y rendimiento hasta colisiones y penetraciones en las mallas.

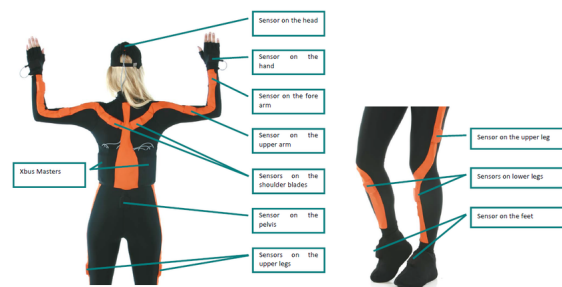


Figura 2.1: Imágen del traje de captura de movimiento XSens ²

2.3. Inteligencia Artificial en videojuegos

2.3.1. Tecnologías existentes

2.3.2. Librerías

2.3.2.1. Pytorch

2.3.2.2. Tensorflow

2.3.2.3. Sentis

2.3.3. Modelos de optimización de telas

2.3.3.1. Subspace Neural Physics: Fast Data-Driven Interactive Simulation

PCA + linear subspace ()

2.3.3.2. Neural Cloth Simulation

Subspace, Recurrent encoder-decoder.

2.3.3.3. PBNS: Physically Based Neural Simulation for Unsupervised Garment Pose Space Deformation

MLP (LBS, relu)

2.3.3.4. HOOD: Hierarchical Graphs for Generalized Modelling of Clothing Dynamics

GNN

²Fuente de la imagen: https://www.researchgate.net/figure/The-Xsens-MVN-full-body-motion-capture-suit-Picture-source-http-wwwxsenscom_fig1_233926874

2.3.3.5. Cloth and Skin Deformation with a Triangle Mesh Based Convolutional Neural Network

Our networks are built using three basic building blocks, namely Convolution, Pooling and Unpooling operators that operate directly on a manifold triangle mesh.

2.3.4. DeepSD: Real-time Cloth Simulation with Deep Learning

CNN. Tambien esta deep wrinkles

Capítulo 3

Descripción del Trabajo

*“In the distant future.
In a land abandoned by man.
Machines wage a continuous war.
Unaware of the futility of their actions.”*
— Nier Automata

3.1. Búsqueda de un dataset

Para poder entrenar modelos de IA se requiere una gran cantidad de datos, en este caso de animaciones de los gestos escogidos que se han mencionado en la introducción. Además, estos datos tienen que estar en un formato en el que se pueda extraer la información de los huesos (posición y rotación) para que pueda ser usado por los modelos, y tenían que ser compatibles con los huesos del traje de captura de movimiento.

3.1.1. Dataset de la Universidad Carnegie Mellon

La Universidad privada Carnegie Mellon, ubicada en Pittsburgh, Pensilvania, tiene disponible de forma gratuita un dataset de movimientos recogidos mediante la captura de movimiento.¹ Sin embargo este dataset no ha sido posible utilizarlo por tres motivos.

El primer motivo es la incompatibilidad del esqueleto utilizado por ellos con el utilizado por el traje de captura de movimiento Perception Neuron 3. Como se puede ver en ? su traje contiene 41 puntos usados para la captura, mostrados en la figura 3.1, dando a lugar al esqueleto mostrado en la misma figura.

¹Enlace a la página del dataset de la Universidad Carnegie Mellon: <https://mocap.cs.cmu.edu/>

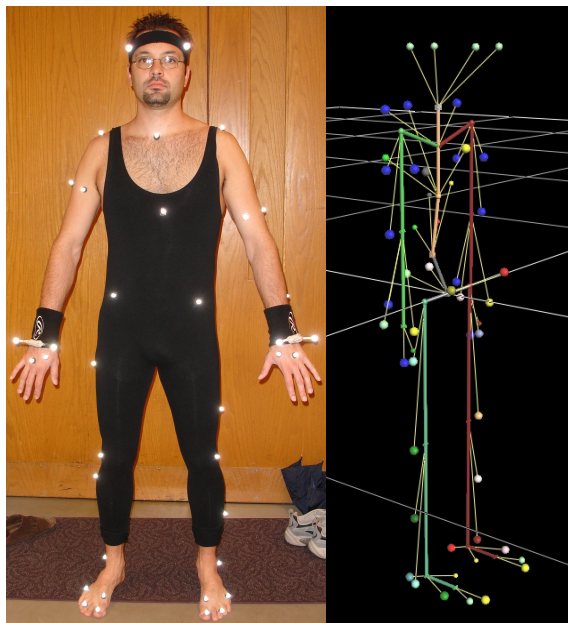


Figura 3.1: Imágen del traje de captura de movimiento utilizado para la base de datos de la Universidad Carnegie Mellon (izquierda) y el esqueleto resultante (derecha). Fuente: <https://mocap.cs.cmu.edu/info.php>

Por otro lado el traje utilizado en este estudio, el Perception Neuron 3, contiene 17 puntos, por lo que debido a las grandes diferencias de los esqueletos decidimos no utilizarlos.

El segundo motivo por el que no se ha usado el dataset de la Universidad Carnegie Mellon es el formato de las animaciones disponibles. Las animaciones presentes en el dataset vienen en tres formatos distintos: c3d, asf (o amc) y vsk (o v). Los formatos c3d, vsk y v son unos formatos en binario usados para objetos en 3D, por lo que no se puede usar de forma sencilla para extraer la información del propio archivo. Por otro lado los archivos asf y amc son archivos en formato ASCII con la información de los huesos. Sin embargo la documentación aportada para poder usar la información es incompleta.

El último motivo por el que no se ha usado este dataset es por la escasez de animaciones de los gestos buscados. Animaciones como “correr” sí están presentes (aunque no en abundancia), pero otras como “pelear” o “saludar” no.

Este último problema también ha estado presente en páginas específicas de bancos de datasets como Kaggle.

3.1.2. Kaggle

Kaggle es una página web dedicada a la ciencia de datos y el aprendizaje automático y la IA. En Kaggle se pueden encontrar datasets, modelos, código y competiciones de IA. Kaggle tiene una gran comunidad de usuarios y una gran selección de datasets para distintas finalidades.

En un primer momento, se pensó en usar datasets de imágenes/vídeo, así que se buscó conjuntos de datos de este formato. La búsqueda no obtuvo los resultados

esperados, ya que no se encontró datos de poses generales como las requeridas, y en concreto no se consiguió mucha cantidad de datos.

Tras analizar las desventajas de usar imágenes y vídeos, se pensó que era mejor usar un conjunto de datos basado en animaciones (o coordenadas de las mismas). Algunas de estas desventajas consideradas fueron:

- Complejidad de renderizado: las imágenes y/o vídeos requerirían de renderizar una cámara externa dentro del motor para capturar la presencia del usuario. Esto ya requeriría más potencia de cómputo, sin contar lo que requeriría el modelo.
- Transferencias de datos más grandes: al plantear un servidor para el modelo, se pensó en el tamaño de las imágenes que habría que mandar para hacer las inferencias. En el caso de tener una red de velocidad limitada sería mejor mandar los puntos concretos del traje que mandar imágenes completas. Las imágenes, incluso al ser de baja calidad, requerirían una cantidad muy grande de píxeles para hacer el modelo del usuario fácilmente reconocible.
- Pérdida de contexto: el traje de captura de movimiento te permite obtener datos tridimensionales de cada punto por defecto, sin necesidad de tener que recrear un esqueleto 3D con capas extra en el modelo.

Tras este cambio de enfoque, se buscó datasets de animaciones y modelos 3D. Resultó ser una búsqueda incluso peor que la anterior, ya que por la limitación de gente que tiene acceso a trajes de captura de movimiento, no había datasets públicos en dataset con este tipo de datos.

Ya que no se encontró un gran dataset que cumpliera con nuestros requerimientos se tomó la decisión de buscar en un banco de animaciones los gestos requeridos y transformar esas animaciones en un formato que pudiesen ser procesados.

3.2. Aplicación final

Una vez los modelos estaban preparados era necesaria una aplicación a modo de demostración que se pueda ejecutar en las gafas de [VR](#) y con el traje de captura de movimiento. La aplicación consta de un panel en el que introduces la IP de la máquina que tiene el servidor de Tensor ejecutándose con el modelo y la aplicación de Axis Studio para poder utilizar el traje. Además, tiene un panel de texto en el que aparecen los resultados de la predicción y un NPC que reacciona a tus gestos. En la figura [3.2](#) se puede ver una captura de esa aplicación.



Figura 3.2: Captura de la aplicación final desde el editor de Unity

3.2.1. Funcionamiento de la aplicación

Al introducir la IP en el cuadro de texto la aplicación se conecta con el traje y el servidor de Tensor. La conexión con el traje es la misma que la del apartado ??, mientras que para el servidor se utiliza una API REST mediante UnityWebRequest,² una API de Unity para comunicarse con servidores web. Esta ejecución se hace mediante una corrutina, la cual al principio espera la cantidad de tiempo óptima entre frame y frame que se determinó en el entrenamiento del modelo usado (en nuestro caso el Random Forest con 0.8 segundos).

La información de los huesos se guarda en una cola, donde se guarda la información actual y, una vez enviado al servidor, se elimina la más antigua, siendo siempre el tamaño de la cola de máximo dos elementos. Al pasar estos segundos la aplicación manda al servidor la información de los huesos en el frame actual y en el frame de hace 0.8 segundos (previamente guardado) en formato JSON mediante POST con protocolo HTTP, a lo que el servidor le responde con dos listas en formato JSON: una con los gestos y otra con el score que ha conseguido cada gesto en la predicción, coincidiendo el índice del score de un gesto con el índice del gesto de la anterior lista. Para hacer todas las conversiones entre cadenas de texto y el formato JSON se ha usado la clase JsonUtility nativa en Unity.

Una vez se tenga la respuesta del servidor empieza de nuevo la corrutina, se ordena de mayor a menor score las predicciones mediante bubble sort para escribirlas en el panel correspondiente y se ejecuta la animación de respuesta del npc. Las animaciones de respuesta que tiene a los gestos se puede ver en la tabla 3.1

²Página de la documentación de Unity: <https://docs.unity3d.com/ScriptReference/Networking.UnityWebRequest.html>

Gesto	Animación de reacción
Bailar	Aplaudir
Saludar	Saludar
Señalar	Mirar
Sentarse	Sentarse
Pelear	Pose defensiva
Correr	Correr

Tabla 3.1: Reacción del npc al gesto predicho del jugador

Si hay un error de conexión en mitad de la ejecución se deberá ingresar de nuevo la IP de la máquina host en el campo de texto para reintentarlo.

En la figura [3.3](#) se puede ver el diagrama de flujo de esta aplicación.

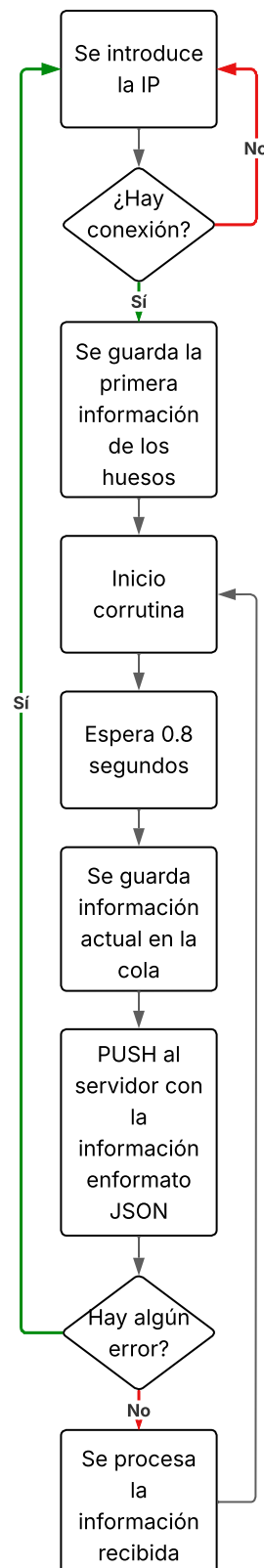


Figura 3.3: Diagrama de flujo de la aplicación de demostración

Conclusiones y Trabajo Futuro

4.1. Conclusiones

Las conclusiones de cada una de las partes del trabajo se presentan en las siguientes secciones de manera más detalladas.

4.1.1. Conclusiones de la búsqueda de datasets

Bla.

4.1.2. Conclusiones de la blablabla

Bla.

4.1.3. Limitaciones

Este estudio ha tenido una serie de limitaciones, desde blablabla Los modelos de redes neuronales no han dado los resultados esperados, esto puede ser por la falta de datos, que aunque haya un número que a simple vista parece suficiente no lo es para entrenar redes neuronales, por las limitaciones de hardware para entrenar modelos más complejos y por los tiempos requeridos para la realización de este estudio no ser lo suficientemente largos como para llevar a cabo entrenamientos más extensos y con mayor búsqueda de hiperparámetros.

Todas estas limitaciones, han llevado al siguiente plan de trabajo a futuro.

4.2. Trabajo Futuro

El trabajo a futuro de este estudio se puede centrar en varios esfuerzos:

- bla
- bla

- bla

Contribuciones Personales

Eva

Muchas cosas.

Pau

Muchas cosas.

Mik

Muchas cosas.

Bibliografía

The sciences, each straining in its own direction, have hitherto harmed us little; but some day the piecing together of dissociated knowledge will open up such terrifying vistas of reality, and of our frightful position therein, that we shall either go mad from the revelation or flee from the deadly light into the peace and safety of a new dark age.

H.P. Lovecraft, *The Call of Cthulhu*

GAMES, E. Chaos cloth. <https://dev.epicgames.com/documentation/unreal-engine/machine-learning-cloth-simulation-overview>, 2025. Accessed: (17/04/2026).

HECKER, C. Physics in computer games (title only). *Communications of the ACM*, vol. 43(7), páginas 34–39, 2000.

JIM HUGUNIN, U. Unite 2016 - gpu accelerated high resolution cloth simulation. <https://www.youtube.com/watch?v=kCGHX1LR318>, 2016. Accessed: (15/04/2026).

KENNY ERLEBEN, K. H., JON SPORRING. *Physics-based Animation*. Charles River Media, 2005.

MACKLIN, M., MÜLLER, M. y CHENTANEZ, N. XPBD: position-based simulation of compliant constrained dynamics. En *Proceedings of the 9th International Conference on Motion in Games*, páginas 49–54. ACM, Burlingame California, 2016. ISBN 978-1-4503-4592-7.

METHOD, V. Obi cloth. <https://assetstore.unity.com/packages/tools/physics/obi-cloth-81333?aid=1011134eQ>, 2019. Accessed: (16/04/2026).

MILLINGTON, I. *Game Physics Engine Development*. CRC Press, 2007. ISBN 9781482267327.

- MÜLLER, M., HEIDELBERGER, B., HENNIX, M. y RATCLIFF, J. Position based dynamics. *Journal of Visual Communication and Image Representation*, vol. 18(2), páginas 109–118, 2007. ISSN 10473203.
- PAUL, P., GOON, S. y BHATTACHARYA, A. History and comparative study of modern game engines. *International Journal of Advanced Computer and Mathematical Sciences*, vol. 3, páginas 2230–9624, 2012.
- TEMPLET, R. M. *Game Physics: An Analysis of Physics Engines for First-Time Physics Developers*. Tesis Doctoral, California State University, Northridge, 2021.
- VA, H., CHOI, M.-H. y HONG, M. Real-time cloth simulation using compute shader in unity3d for ar/vr contents. *Applied Sciences*, vol. 11(17), 2021. ISSN 2076-3417.

Carteles para la generación del dataset



Figura A.1: Cartel colgado en la facultad de informática

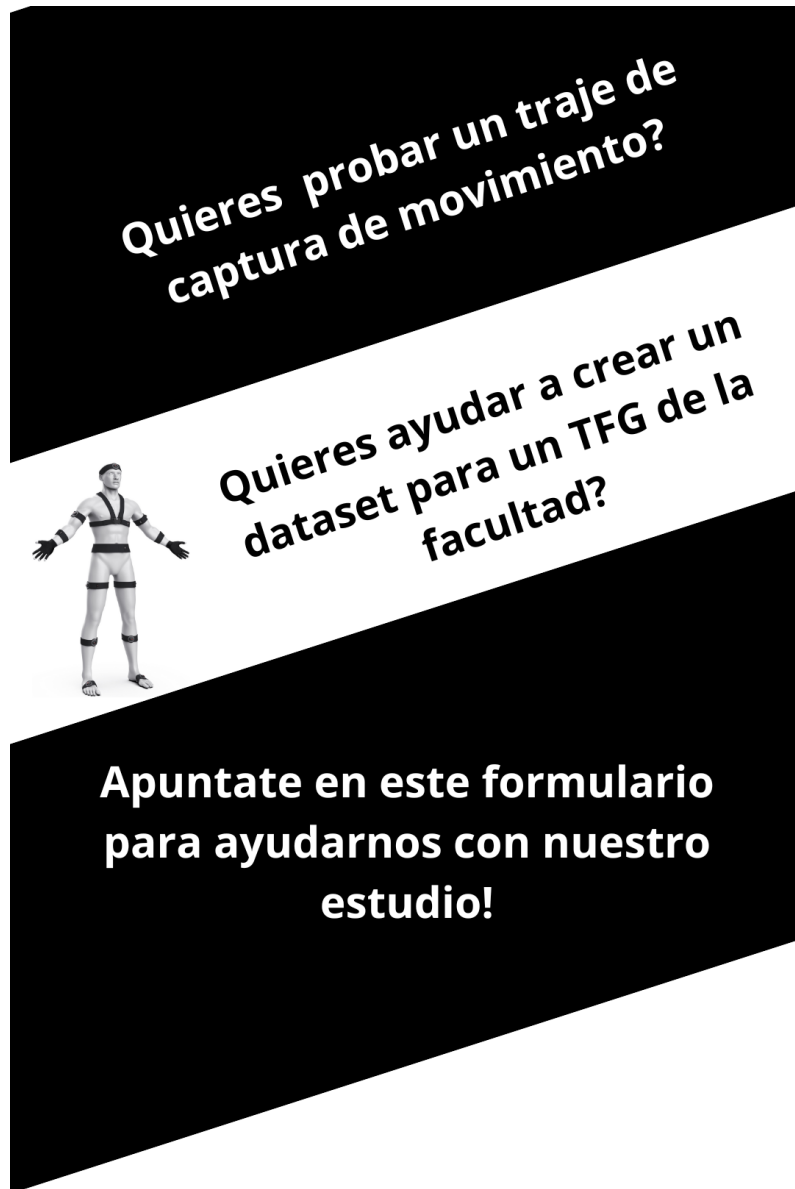


Figura A.2: Cartel colgado en redes sociales



Figura A.3: Cartel colgado en pantallas de la facultad

Formulario de recogida de citas

Participación Dataset para detección de poses

Necesitamos datos para un TFG de la Facultad de Informática de la UCM. Lo importante, **¿qué datos recopilamos?** La siguiente lista de datos son los datos que vamos a recopilar y hacer públicos en el propio estudio del tfg o en un dataset público en [kaggle](#):

- Edad
- Sexo
- Nacionalidad
- Datos de coordenadas de distintas partes del cuerpo dadas por un traje de captura de movimiento

Estos datos se recogerán en un formulario a parte el día de la prueba, siendo hechos desde un correo de los organizadores y permaneciendo totalmente anónimos sin correlación a la persona que ha realizado la prueba.

OBJETIVO DE LOS DATOS: los datos de edad, nacionalidad y sexo son meramente estadísticos, solo se usarán para demostrar la heterogeneidad (o falta de) de los datos tomados. Los datos de coordenadas se usarán para entrenar distintos modelos de clasificación de pose con la intención de poder decir con acierto el gesto o pose que está haciendo una persona mientras lleva el traje puesto.

LOCALIZACIÓN DE LA PRUEBA: [Facultad de Informática \(UCM\)](#), Calle del Prof. José García Santesmases, 9, Moncloa - Aravaca, 28040 Madrid

alejba02@ucm.es [Cambiar de cuenta](#)

* Indica que la pregunta es obligatoria

Correo *

Tu dirección de correo electrónico 

¿Aceptas la recogida de datos y la participación en el muestreo? *

☐ Si

☐ No

Siguiente

Página 1 de 2

Borrar formulario

Este formulario se creó en Universidad Complutense de Madrid.
¿Parece sospechoso este formulario? [Informe](#)



Figura B.1: Formulario de recogida de citas (primera parte)

Participación Dataset para detección de poses

alejba02@ucm.es [Cambiar de cuenta](#)

Horario disponible

A continuación marca las horas en las podrías estar disponible. Con la respuesta que proporciones te mandaremos un correo con un día y hora determinados.

Lunes

- ☐ 9:00-10:00
- ☐ 10:00-11:00
- ☐ 11:00-12:00
- ☐ 12:00-13:00
- ☐ 13:00-14:00
- ☐ 14:00-15:00
- ☐ 15:00-16:00
- ☐ 16:00-17:00
- ☐ 17:00-18:00
- ☐ 18:00-19:00
- ☐ 19:00-20:00

Figura B.2: Formulario de recogida de citas (segunda parte)

Resultados de los distintos modelos CNN

La línea verde en los gráficos de tensorboard indican la mejor combinación de hiperparámetros encontrada en cada caso

C.1. Intervalo 0.2s

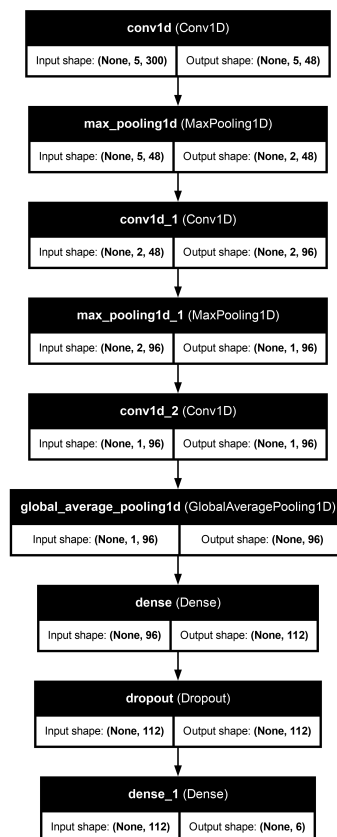


Figura C.1: Esquema del modelo CNN con 0.2s de intervalo

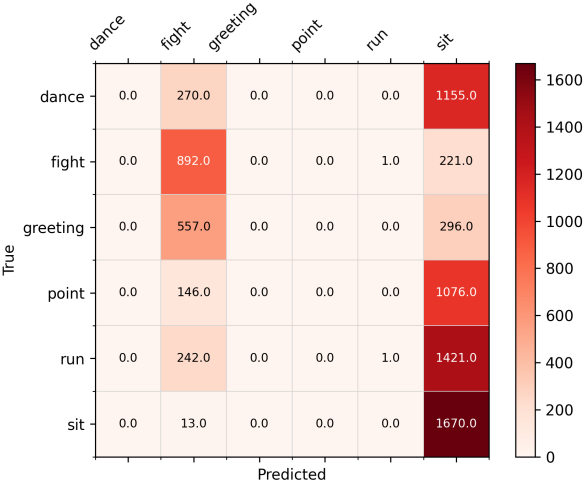


Figura C.2: Matriz de confusión del modelo CNN con 0.2s de intervalo

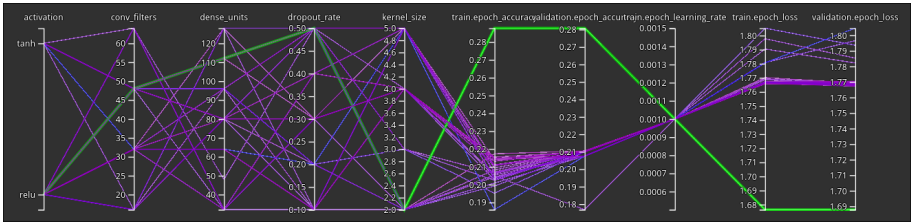


Figura C.3: Gráfico de entrenamiento del modelo CNN con 0.2s de intervalo (mejor val_accuracy = 0.28777)

C.2. Intervalo 0.4s

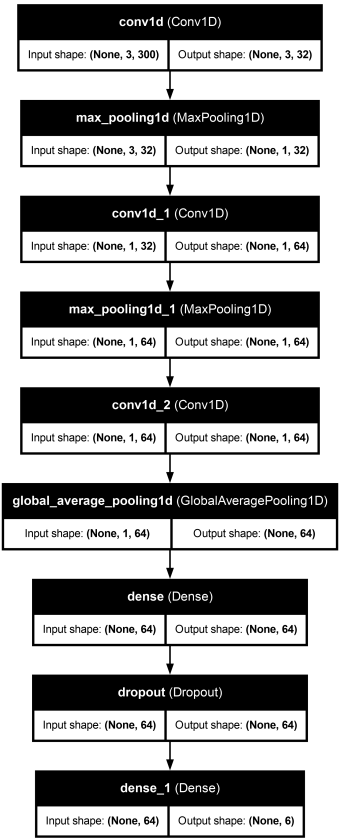


Figura C.4: Esquema del modelo CNN con 0.4s de intervalo

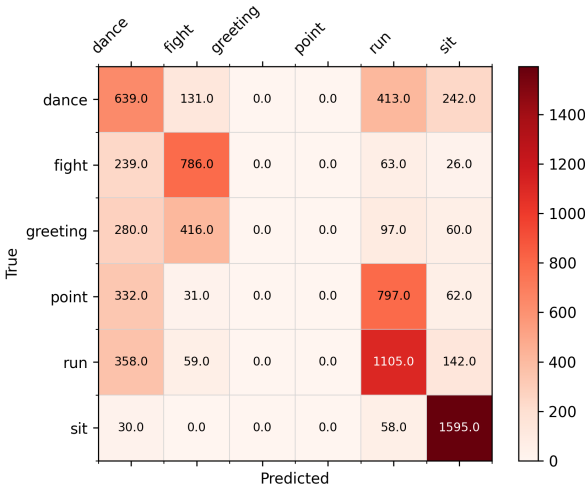


Figura C.5: Matriz de confusión del modelo CNN con 0.4s de intervalo

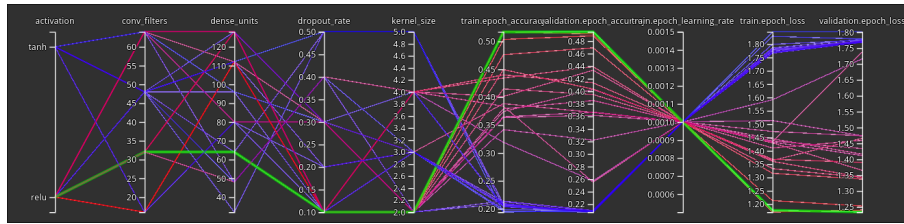


Figura C.6: Gráfico de entrenamiento del modelo CNN con 0.4s de intervalo (mejor val_accuracy = 0.49473)

C.3. Intervalo 0.6s

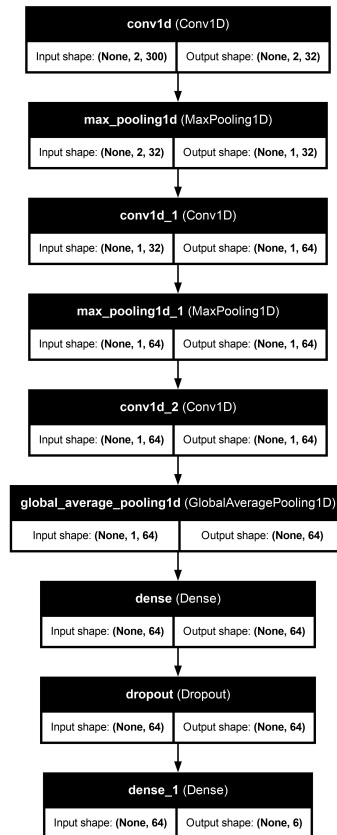


Figura C.7: Esquema del modelo CNN con 0.6s de intervalo

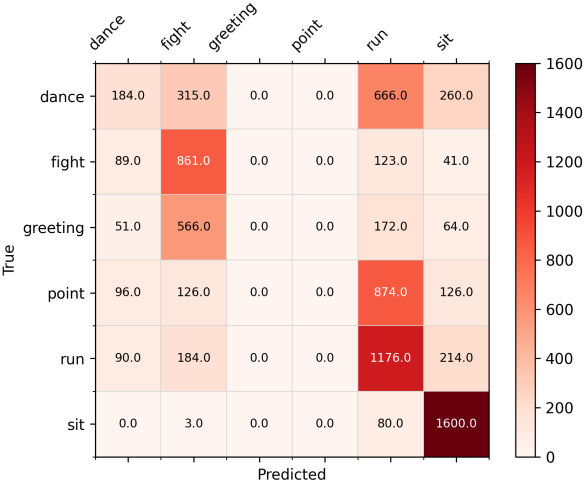


Figura C.8: Matriz de confusión del modelo CNN con 0.6s de intervalo

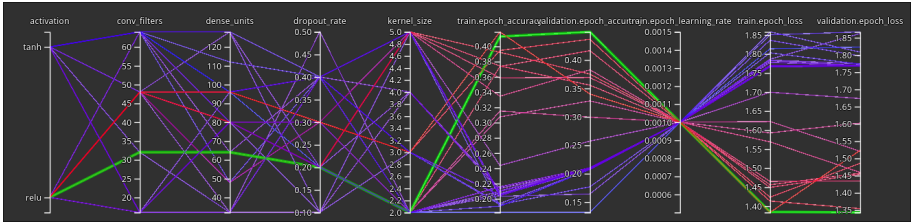


Figura C.9: Gráfico de entrenamiento del modelo CNN con 0.6s de intervalo (mejor val_accuracy = 0.44952)

C.4. Intervalo 0.8s

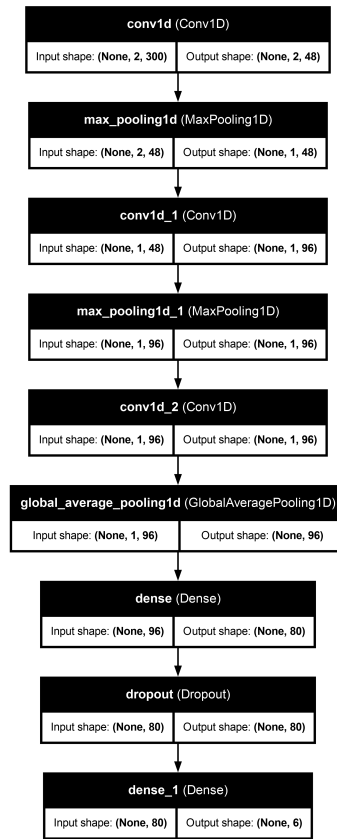


Figura C.10: Esquema del modelo CNN con 0.8s de intervalo

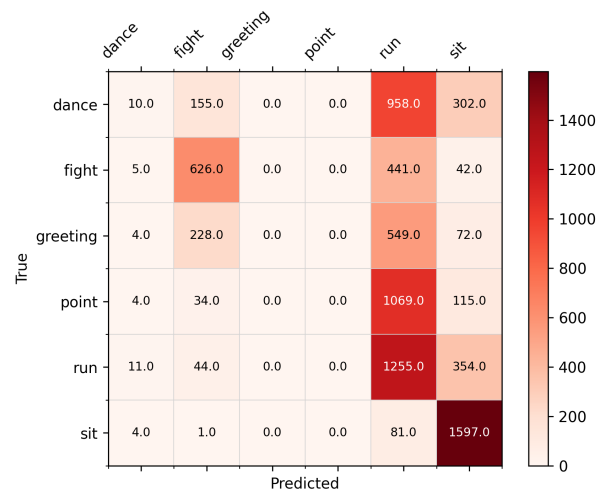


Figura C.11: Matriz de confusión del modelo CNN con 0.8s de intervalo

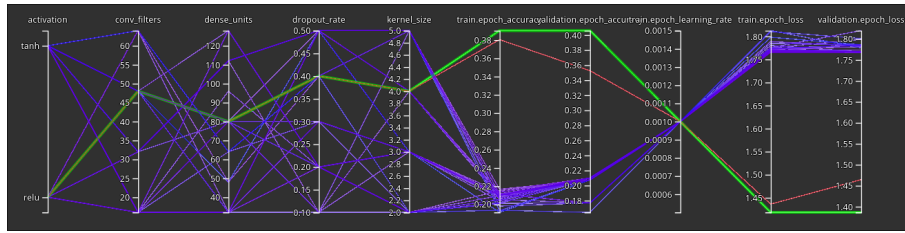


Figura C.12: Gráfico de entrenamiento del modelo CNN con 0.8s de intervalo (mejor val_accuracy = 0.40583)

C.5. Intervalo 1.0s

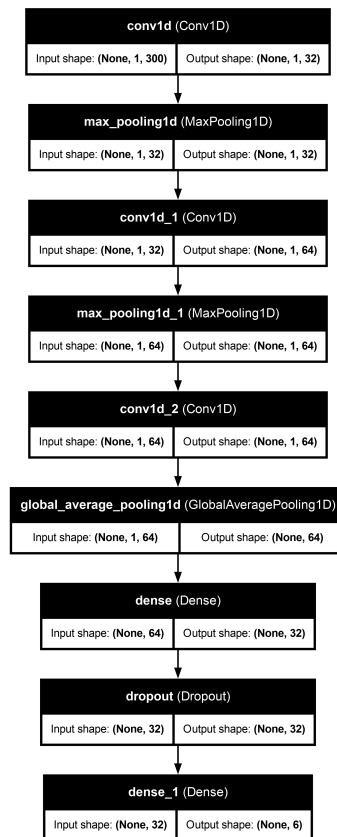


Figura C.13: Esquema del modelo CNN con 1.0s de intervalo



Figura C.14: Matriz de confusión del modelo CNN con 1.0s de intervalo

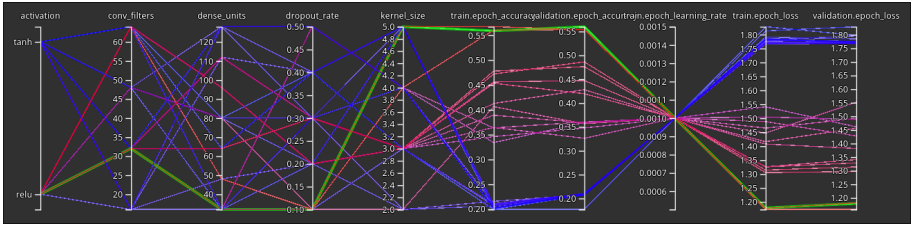


Figura C.15: Gráfico de entrenamiento del modelo CNN con 1.0s de intervalo (mejor val_accuracy = 0.56002)

Glosario

CNN Convolutional Neural Network

IA Inteligencia Artificial

VR Realidad Virtual

Licencia de uso del documento

Esta obra está bajo una licencia [Creative Commons](http://creativecommons.org/licenses/by-sa/4.0/legalcode) «Atribución-NoComercial-CompartirIgual 4.0 Internacional».



<http://creativecommons.org/licenses/by-sa/4.0/legalcode>.

Licencia de uso del código fuente

Los archivos de código fuente para generar este documento se encuentran en <https://github.com/FratosVR/Memoria-TFG> bajo la licencia [GPL-3.0](#)